



必要機能の一覧と実装の対応表

CHIZUBA (チズバ)

| CODIHA 2026 ハッカソン部門

| チーム 愛で千葉は救えるのか

| 説明資料②

- ・「必要機能」は説明資料①（サービスの概要）2 ページ目と同じ 8 項目・同じ番号。
- ・「対応する実装」は、提出した作業ディレクトリからの相対パスとシンボル名。行番号はコードを 1 行足すだけでずれるので使っていない。全行のパスとシンボルは docs/presentation/submission/verify_table.py が機械的に照合している
- ・備考の末尾に「確かめ方」を書いた。起動した審査員が その場で 30 秒で動作を確認できる手順になっている
- ・ログインの仕組みは、すぐ下の囲みにまとめた。各行の「確かめ方」は、審査環境（認証キー未設定）でデモログインをした状態を前提に書いてある

ログインについて — 「デモログイン」は本人確認を省いた入り方という意味で、機能の制限ではない

- ・ログインがしているのは 2 つだけ。「誰が投稿したか」を記録することと、一般ユーザーか行政ユーザーかの役割を決めること、の 2 つだけである。地図・ハザードマップ・投稿の閲覧・投稿一覧・CSV / GeoJSON の書き出しは、ログインなしで全部できる。ログインが要るのは投稿とコメントのときだけ
- ・ログイン方法は 2 つある。① Google ログイン（本人確認を Google に任せる）と、② デモログイン（表示名を入れ、一般ユーザーか行政ユーザーかを選ぶだけ）
- ・2 つの違いは本人確認をするかないかだけで、ログインした後にできることはまったく同じ。投稿・写真の添付・コメント・行政ユーザーとしての対応状況の更新と公式回答まで、どちらで入っても同じように動く。デモログインだから使えない機能というものは無い
- ・上の表の 8 機能は、すべてデモログインだけで確認できる。各行の「確かめ方」のとおり操作すれば、行政ユーザーの操作まで含めて全部その場で動く
- ・Google の認証キーがある環境では、同じログイン画面に「Google でログイン」が足されるだけで、デモログインは消えない。クライアントシークレットは秘密情報なので提出物に同梱していないため、審査環境ではデモログインだけが表示される。Google ログインは審査環境では検証できないので、この表の行にしていない

番号	必要機能	対応する実装（どのファイルのどこか）	備考と、確かめ方
1	ハザードマップの表示	src/lib/hazards.ts ... HAZARDS (4 種のタイル URL)・ HAZARD_LEGENDS (凡例) src/components/MapView.tsx ... raster ソースとレイヤーの積み上げ src/components/HazardLegend.tsx ... 凡例の描画 src/components/ControlPanel.tsx ... 種類ごとの ON/OFF と不透明度	実装済 洪水・高潮・津波の浸水想定と土砂災害警戒区域（急傾斜地の崩壊）の 4 種。タイルは国土交通省「重ねるハザードマップ」で、ブラウザが直接取得する。起動直後は洪水だけ ON（土砂災害は指定が北部に偏るため既定 OFF） 確かめ方: 左の操作パネルを一番下まで送る → 「土砂災害（急傾斜地の崩壊）」を ON → 北部の斜面に赤と黄の帯が出て、凡例が 4 段階増える
2	市川市オープンデータの重ね合わせ	src/lib/layers.ts ... LAYERS (データの場所・色・ポップアップ項目) src/lib/scenic.ts ... SCENIC_CATEGORIES (景観 100 選の色分け) public/data/evacuation_sites.geojson (123) public/data/aed_locations.geojson (304) public/data/childcare_facilities.geojson (388) public/data/scenic_spots.geojson (100) src/components/MapExplorer.tsx ... 取得 / MapView.tsx ... 描画	実装済 指定緊急避難場所 123・AED 設置箇所 304・子育て施設 388・いちかわ景観 100 選 100。すべて市川市オープンデータ (CC BY 4.0) 由来で、DB を経由せず同梱してある（データベースが落ちていても出る） 確かめ方: 操作パネル「表示するデータ」で「AED 設置箇所 304 件」を ON / OFF → 点が出入りする。点を押すと名称・所在地が出る
3	危険箇所の市民報告	src/lib/reports.ts ... REPORT_CATEGORIES のカテゴリ hazard src/app/api/reports/route.ts ... POST (投稿の受け口) src/lib/reportInput.ts ... parseReportForm (検証) src/components/ReportForm.tsx ... 投稿フォーム src/components/ReportPanel.tsx ... 詳細パネル (写真・本文・コメント) src/components/ReportEditForm.tsx ... 投稿者本人による本文の編集	実装済 位置+写真 (3 枚まで) + 説明で投稿でき、地図にピンが出る。投稿の市町村は座標から決めるので詐称できない。写真は先頭バイトまで確かめる 確かめ方: 右上「ログイン」→ 表示名を入れて「デモログイン」→ 操作パネル「危険箇所を投稿する」→ 地図で位置を指定して送信 → 橙のピンが増える
4	浸水報告と、投稿時点の雨量の自動記録	src/lib/reports.ts ... REPORT_CATEGORIES のカテゴリ flood src/lib/jma.ts ... observeRainfall (アメダス実況の取得とキャッシュ) src/app/api/reports/route.ts ... 投稿時にサーバー側で雨量を焼き込む src/components/FloodRainfall.tsx ... 観測所名と距離つきの表示	実装済 投稿した瞬間の 1 時間降水量が気象庁アメダス実況から自動で記録される。投稿者は入力・変更できない。最寄りの観測所の値であってその地点の実測値ではないため、観測所名と距離を必ず併記する（市川市の中心からは船橋・約 10 km）。過去に浸水報告のある地域への注意表示は、気象庁の予報で降水確率が 30% 以上のときだけ出る（浸水の予測はしない） 確かめ方: ログインして「浸水を投稿する」→ 送信 → 詳細パネルに「投稿時の雨量 ○ mm/h / 観測所名と距離」が入る（例:

番号	必要機能	対応する実装（どのファイルのどこか）	備考と、確かめ方
		src/lib/weather.ts ... buildFloodAlert (注意表示を出すかの判定) src/components/FloodAlertCard.tsx ... 注意表示	船橋アメダス 約 10 km。最寄りの観測所は投稿位置によって変わる
5	観光マップ（景観 100 選と徒歩ナビ）	src/lib/mapModes.ts ... MAP_MODES （防災／観光で出す組） src/components/MapModeTabs.tsx ... タブの切り替え src/lib/scenicPhotos.ts ... SCENIC_PHOTOS （54 か所の写真と出典） src/app/api/routing/route.ts ... OSRM への中継（1 秒 1 リクエスト） src/components/RouteCard.tsx ... 距離と所要時間の表示	実装済 景観スポット 100 か所を日本語と英語の解説つきで表示し、100 か所のうち 54 か所に写真が付く（作者とライセンスを併記）。そのまま徒歩ナビの目的地にできる。経路サービス（OSRM）が落ちたら直線距離と徒歩 4.8 km/h の概算に自動で切り替わり、概算である旨を画面に出す（無反応で止まらない） 確かめ方 : ヘッダー直下「観光マップ」→ 景観スポットの点を押す → 解説と写真 → 「ここへナビ」→ 経路と距離・所要時間が出る
6	観光おすすめの市民投稿	src/lib/reports.ts ... REPORT_CATEGORIES のカテゴリ spot src/app/api/reports/route.ts ... 危険箇所・浸水と 同じ 1 経路 src/components/ReportForm.tsx ... 同じ 1 フォーム	実装済 景観・お土産・飲食のおすすめを住民と行政の両方が投稿できる。3 種類の投稿を 1 テーブル・1 API・1 フォームに統一しており、違いは category と details (jsonb) だけ 確かめ方 : 観光マップでログイン → 「観光おすすめを投稿する」 → 送信 → 赤紫のピンが増える
7	行政からの応答	src/app/api/reports/[id]/route.ts ... PATCH （対応状況の更新） src/app/api/reports/[id]/comments/route.ts ... POST （公式コメント） src/lib/reports.ts ... REPORT_STATUSES （4 段階の定義） src/components/ReportStatusControl.tsx ... 更新の UI src/components/OfficialBadge.tsx ... 行政の投稿・発言の区別表示	実装済 公式コメントと対応状況 4 段階（未対応／受付／対応中／対応済）の更新。更新できるのは担当する市町村の投稿だけで、一般ユーザーが変えようとする HTTP 403 で断る 確かめ方 : ログイン画面で「行政ユーザー（市川市）」を選ぶ（認証キー未設定の審査環境では PIN なしでそのまま選べる）→ 投稿を開くと「行政の対応状況を更新する」が出る
8	投稿の一覧・絞り込みと、オープンデータとしての書き出し	src/app/reports/page.tsx ... 一覧の画面（/reports） src/lib/reportRange.ts ... RANGE_PRESETS ・ normalizeRange （期間） src/lib/searchText.ts ... normalizeSearch （NFKC + 小文字でそろえる） src/lib/reportExport.ts ... toCsv ・ toGeoJson src/app/api/reports/export/route.ts ... GET （書き出しの受け口）	実装済 カテゴリ・期間・キーワードで絞り込み、絞り込んだそのままの条件で書き出せる。CSV は UTF-8 BOM + CRLF + RFC 4180, GeoJSON は RFC 7946。出すのは画面で誰でも読める項目だけで、アカウントの情報は含めない。デモ投稿の雨量は観測値ではないので書き出さない 確かめ方 : ヘッダー「投稿一覧」→ 「7 日間」を押す → 件数が変わる → 「CSV」を押すと その条件のまま落ちてくる

この表の確かめ方

「対応する実装」に書いたファイルとシンボルは、全行を機械的に照合してある（docs/presentation/submission/verify_table.py。パスの存在と、シンボルがそのファイルに実在するかを確かめ、1 つでも欠ければ非 0 で終わる）。パスは提出した作業ディレクトリからの相対（例: src/lib/hazards.ts は ai-de-chiba-map/src/lib/hazards.ts）。docs/で始まるものだけは開発リポジトリ側を指していて、提出物には含まれない。

実装状況

上の 8 機能はすべて実装済みで、展開した提出物を起動して動作を確認している。